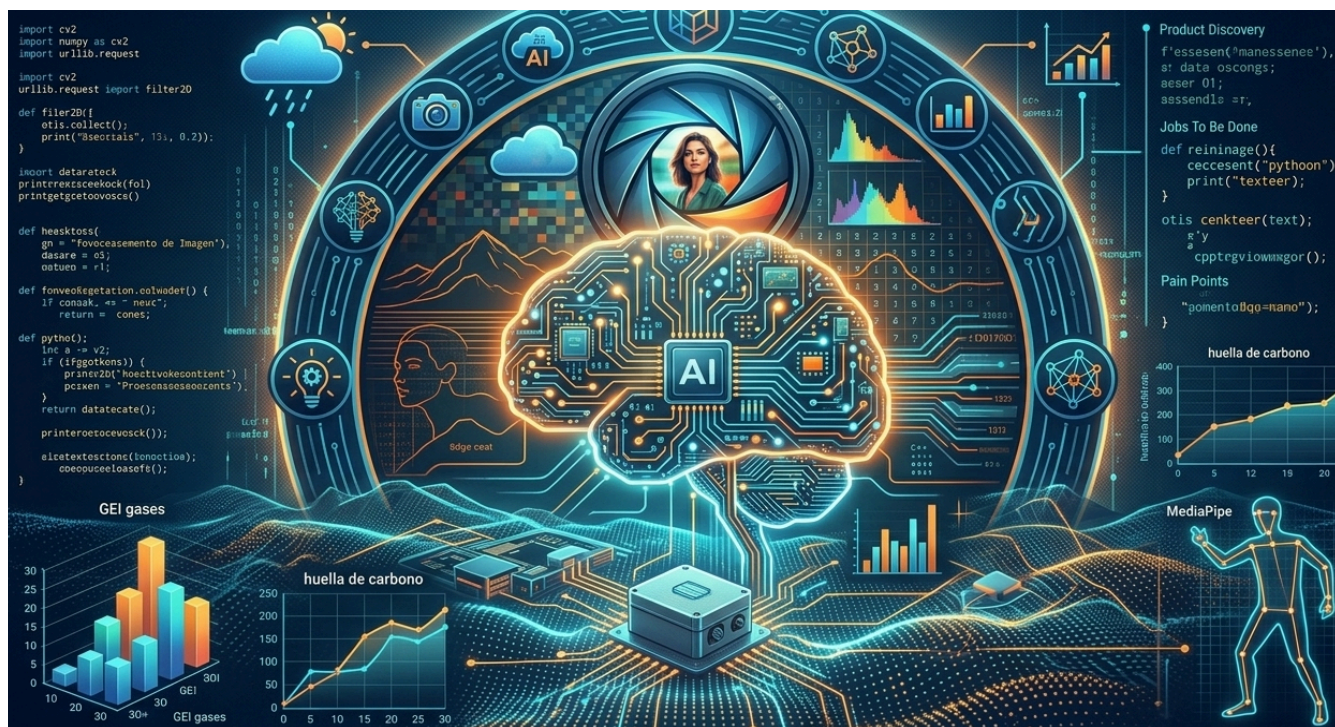


Tecnicatura:

TECNICATURA SUPERIOR EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL



|Primer Cuatrimestre - 3er Año

|MÓDULO - Procesamiento de IMÁGENES

|Docente - CHARLETTI, Carlos Ignacio

|Grupo - VisioNet

|Repositorio Grupal: <https://github.com/Nieves862/-ISPC-Procesamiento-Imagenes-2026>

// Evidencia Final // Google Colab: [EvidenciaIII_FINAL_CCSN_a_YOLO.ipynb](#)

Integrantes del Grupo:

• Guaraz Emanuel	38.276.061
• Direni Carlos	28.117.281
• Testa Andrea Paola	25.442.630
• Villalba Valeria Nieves	26.970.862
• Allende Olmedo Nicolás	35.871.057
• García Carlos	40.518.523
• Moreno Raúl	29.201.107



Entrega Final: Implementación del Modelo y Análisis de Resultados

YOLO: DETECCIÓN Y CLASIFICACIÓN DE NUBES Y CIELO DESPEJADO

Adaptación de YOLOv8 para detección y clasificación de nubes

En esta sección adaptamos el modelo YOLOV8Detector de KerasCV para que su capa de salida esté configurada para las 4 clases de nubes que hemos definido (CLASS_NAMES_TF).

Es importante recordar que YOLO es un modelo de **detección de objetos**, lo que significa que no solo clasifica una imagen completa, sino que también localiza objetos dentro de ella mediante **cajas delimitadoras (bounding boxes)**. Para entrenar un modelo YOLO de manera efectiva para detectar y clasificar nubes con estas 4 categorías, idealmente necesitaríamos un dataset de imágenes donde cada nube (o cada instancia de nube si hay varias en una imagen) esté anotada con una caja delimitadora y su clase correspondiente.

El backbone (yolo_v8_m_backbone) se inicializa con pesos pre-entrenados en el dataset "Cloud.V3i.yolov8", lo que puede ser un buen punto de partida. Sin embargo, para un rendimiento óptimo en la detección de nubes, este modelo (o al menos su capa de detección) deberá ser **fine-tuneado** con un dataset más voluminoso de nubes anotado con bounding boxes.

NOTA: El **Dataset** utilizado tiene un origen aparentemente Asiático, por lo que los nombres de las nubes detectadas figuran en su idioma natal, a menos que no lo encuentren o no esté etiquetada (la Nube) de esa manera, el modelo optará por una de las clasificaciones aportadas por el equipo.



Bloque 1: Configuración del Entorno de Ejecución y Gestión de Dependencias

Descripción: Este bloque inicial se encarga de acondicionar el entorno virtual de Google Colab, garantizando la disponibilidad de todo el ecosistema de librerías necesarias para el procesamiento masivo de datos y el entrenamiento de modelos de Deep Learning.

El script ejecuta tres acciones principales:

1. **Instalación de Dependencias Críticas:** Descarga de forma silenciosa (-q) el motor oficial de Inteligencia Artificial **ultralytics** (YOLOv8), las herramientas de transferencia de datasets (**kagglehub**), y frameworks avanzados de Computer Vision como **keras-cv** y **tensorflow**.
2. **Carga del Stack Tecnológico:** Importa los módulos esenciales para la manipulación matricial de imágenes (**numpy**, **OpenCV**, **PIL**), la visualización de datos (**matplotlib**) y la arquitectura de redes neuronales convolucionales de **TensorFlow / Keras** (incluyendo capas nativas y la red preentrenada **MobileNetV2** para flujos comparativos).
3. **Auditoría de Hardware:** Realiza un control de calidad al verificar la versión de TensorFlow activa y validar de forma autónoma la correcta aceleración por hardware mediante la **GPU (Nvidia T4)** del entorno, asegurando que el pipeline se ejecute a máxima velocidad.

BLOQUE 2: Carga y Preparación del Dataset YOLO desde Google Drive

Descripción: Este bloque automatiza el aprovisionamiento de datos en la instancia local de Google Colab. En lugar de realizar una lectura directa o una copia carpeta por carpeta desde el almacenamiento virtual de Google Drive (procesos que suelen generar cuellos de botella y lentitud en entornos de computación distribuidos), el script descarga de forma eficiente el dataset empaquetado en formato comprimido (.zip) y lo procesa localmente. Esto maximiza la velocidad de lectura/escritura del disco virtual durante el entrenamiento de YOLOv8.

Acciones Principales del Script:

- **Enlace y Autenticación de Drive:** Establece la conexión de infraestructura mediante el método `drive.mount`, vinculando el entorno de ejecución temporal de Colab con la unidad de almacenamiento en la nube del usuario que ejecuta el notebook.
- **Descarga Eficiente Automatizada (gdown):** Utiliza la utilidad de nivel de sistema `gdown` para descargar directamente el archivo comprimido del dataset (`cloud.v3i.yolov8.zip`) empleando su identificador único de archivo corporativo (`FILE_ID`). El script incluye una lógica de verificación defensiva (`os.path.exists`) para omitir la descarga si el archivo ya

se encuentra en la memoria temporal, optimizando el consumo de ancho de banda y los tiempos de desarrollo.

- **Extracción y Descompresión en Caché Local:** A través del módulo nativo zipfile, el software extrae los arreglos de imágenes (subcarpetas de entrenamiento, validación y testeo) y los archivos de configuración indexados dentro del directorio local de alta velocidad /content/dataset. Al igual que en el paso de descarga, cuenta con un control de redundancia que evita la re-extracción innecesaria si la estructura ya está validada, dejando el pipeline listo (DATASET_DIR) para los módulos de visión artificial del equipo VisioNet.

PASO 1: VERIFICACIÓN DE DESCOMPRESIÓN DE ARCHIVO .ZIP CON DATASET:

Descripción: Este bloque realiza una auditoría de control de calidad sobre el directorio local de alta velocidad para verificar que el proceso de descompresión del archivo .zip se haya ejecutado correctamente. El script escanea el entorno, mapea la estructura de subdirectorios nativa de YOLOv8 (identificando las particiones esenciales de train, valid y test) y calcula el volumen total de datos disponibles para la fase de entrenamiento del equipo VisioNet.

Acciones Principales del Script:

- **Mapeo Dinámico de Subcarpetas:** A través del módulo os, el sistema lista y filtra únicamente los directorios físicos generados en la raíz de la extracción. Esto permite validar visualmente que los conjuntos de datos fragmentados en formato YOLO se encuentren presentes y perfectamente accesibles para los algoritmos.
- **Enrutamiento Inteligente y Defensivo:** El código implementa una lógica condicional de adaptabilidad jerárquica. Primero busca la ruta estándar y óptima de Ultralytics (dataset/train/images). Si el dataset fue exportado con una estructura directa o simplificada, el script lo detecta de forma autónoma y reasigna el puntero directamente a la carpeta raíz de entrenamiento (dataset/train), evitando que el cuaderno sufra interrupciones o errores de tipo FileNotFoundError en los bloques siguientes.
- **Conteo de Activos Visuales:** Una vez consolidada la ruta definitiva de las imágenes de entrenamiento, el software cuantifica el volumen exacto de archivos (total_imgs) que ingresarán al pipeline de procesamiento digital e inferencia artificial, dejando el entorno auditado y listo para la fase de análisis morfológico en el proyecto Eco-Intelligence.

PASO 2: INSPECCIONAR LA ESTRUCTURA INTERNA DEL DATASET DESCOMPRESO: Pipeline de Procesamiento Digital de Imágenes (PDI) Avanzado

Descripción: Este bloque constituye el núcleo de la ingeniería de características visuales (Feature Engineering) del proyecto. Mediante el uso estratégico de la librería OpenCV, se diseña y ejecuta un pipeline secuencial de procesamiento sobre muestras aleatorias del dataset. El objetivo principal es aislar el ruido cromático del cielo y maximizar los gradientes morfológicos de las masas de vapor, transformando los datos crudos en matrices optimizadas para el posterior análisis y entrenamiento de los modelos de visión artificial en el marco del proyecto Eco-Intelligence.

Acciones Principales del Script:

- **Decodificación Espacial (BGR a RGB):** OpenCV carga las imágenes de forma nativa en el espacio de color BGR. El script realiza la transposición matricial inmediata al estándar RGB mediante `cv2.cvtColor` para garantizar una visualización cromática realista y exacta en los reportes de control y auditoría del equipo.
- **Reducción Dimensional (Escala de Grises):** Transforma la matriz tridimensional (canales de color R, G, B) a una representación bidimensional de intensidades lumínicas. Este paso metodológico elimina la información redundante del color del fondo atmosférico (azul/celeste) y reduce drásticamente el costo computacional de las operaciones matemáticas subsiguientes.
- **Optimización de Contraste (Ecuilización de Histograma):** Aplica el algoritmo `cv2.equalizeHist` para redistribuir uniformemente las intensidades de los píxeles grises. En la meteorología orientada a visión computacional, esta técnica es clave: expande el contraste en zonas críticas, permitiendo resaltar microtexturas, densidades y variaciones de vapor en formaciones nubosas que originalmente presentaban una iluminación plana o sobreexpuesta.
- **Segmentación Estructural (Desenfoque Gaussiano + Canny):**

- Suavizado: Se aplica un filtro Gaussiano con un kernel de (5, 5) para disipar el ruido de alta frecuencia del sensor fotográfico y homogeneizar las regiones.
- Detección de Bordes: El algoritmo de Canny calcula las derivadas espaciales de la imagen para aislar discontinuidades abruptas de brillo. Esto permite mapear y segmentar los contornos y límites geométricos de las estructuras nubosas, abstrayendo su morfología base para el grupo VisioNet.
- **Matriz de Control Visual (Gridspec):** Estructura un cuadro comparativo cuádruple en tiempo real mediante plt.subplot. Esta matriz sirve como una robusta herramienta de auditoría visual para la entrega de evidencias técnicas, validando paso a paso la efectividad de cada etapa del pipeline de procesamiento.

PASO 3: INFERENCIA YOLOv8 CON CONTROL DE BORDES e IMÁGENES VACÍAS

Pipeline de Inferencia, Control de Bordes y Manejo de Cielos Limpios

Descripción: Este bloque implementa un motor de renderizado y auditoría visual de etiquetas en formato YOLO (Ground Truth). El script está diseñado bajo un enfoque defensivo para realizar dos tareas críticas en el control de calidad de los datos del proyecto Eco-Intelligence: la identificación automática de imágenes sin objetos (*Background Images*) para mitigar falsos positivos, y el cálculo geométrico preciso para evitar el desbordamiento de texto en los límites del lienzo (*Bounding Box Overflow*) del equipo VisioNet.

Acciones Principales del Script:

- **Sincronización Dinámica de Metadatos (YAML):** Carga el archivo de configuración `data.yaml` para mapear los índices numéricos de las clases a sus nombres meteorológicos oficiales. Cuenta con un arreglo de respaldo (*fallback*) estructural para asegurar la continuidad del pipeline informático ante posibles fallos de lectura en el almacenamiento distribuido.
- **Evaluación Condicional de Escenario (Árbol de Decisión):**

- **Caso A (Cielo Limpio):** Detecta de forma autónoma archivos de anotación de longitud cero. El sistema interpreta la ausencia de vectores como una imagen de fondo (esencial para enseñarle a la IA a no generar falsas alarmas con el cielo despejado) y genera un recuadro verde perimetral con un cartel de aviso en la zona superior.
- **Caso B (Nubes Catalogadas):** Procesa de forma secuencial las líneas del archivo `.txt`. Desnormaliza las coordenadas relativas de YOLO multiplicándolas por las dimensiones de píxeles reales de la pantalla para reconstruir de forma exacta los límites absolutos de las cajas contenedoras de las nubes.
- **Algoritmo de Control de Bordos Crítico:** Evalúa mediante la función matemática `cv2.getTextSize` el espacio físico que ocupará el texto de la etiqueta. Si la nube detectada se encuentra pegada al techo del mapa (haciendo que el texto quede fuera de la matriz visual), el script invierte dinámicamente la posición del recuadro azul hacia el interior de la caja, garantizando lecturas perfectas en la interfaz meteorológica.



PASO 3-1 Lanzamiento del Entrenamiento de la Red Neuronal (YOLOv8)

Descripción: Este bloque ejecuta la fase medular del proyecto integrador: la transferencia de aprendizaje (*Transfer Learning*) sobre una arquitectura de red neuronal convolucional profunda de vanguardia. Utilizando el modelo base preentrenado YOLOv8 Nano (seleccionado por su alta eficiencia en entornos embebidos y de cómputo perimetral), el sistema ajusta sus capas sinápticas para especializarse en la detección, delimitación y

clasificación morfológica de las formaciones nubosas en el marco de la estación meteorológica Eco-Intelligence.

Acciones Principales del Script:

- **Instanciación del Modelo Base:** Carga la topología de red profunda [yolov8n.pt](#). Mediante la técnica de transferencia de aprendizaje, la IA aprovecha la capacidad previa de extracción de características complejas en imágenes genéricas y las redirige hacia el dominio específico de la meteorología visual.
- **Parametrización del Pipeline de Entrenamiento (`model.train`):** Configura los hiperparámetros críticos del algoritmo de aprendizaje:
 - *Ciclos y Lotes:* Fija el entrenamiento en **30 épocas** de optimización con un tamaño de lote (*batch*) de **16 muestras** simultáneas, equilibrando de forma óptima los tiempos de convergencia.
 - *Dimensión Matricial:* Estandariza la entrada de los sensores visuales a una resolución de **640 píxeles** (`imgsz=640`).
 - *Aumento de Datos (Augmentation):* Habilita técnicas de distorsión y ajuste cromático aleatorio sobre los arreglos de píxeles en tiempo de ejecución. Esto previene el fenómeno de sobreajuste (*overfitting*), obligando a la red a generalizar las características morfológicas reales de las nubes.
- **Auditoría Estructural y Resguardo de Pesos:** El script verifica el direccionamiento hacia los metadatos del archivo [data.yaml](#) y coordina el uso de los recursos de hardware distribuidos de **VisionNet**, emitiendo alertas defensivas en caso de fallas y resguardando de manera automática los archivos de coeficientes entrenados ([best.pt](#)) para su despliegue final.

PASO 3.2: Módulo de Inferencia en Vivo e Interfaz Adaptativa (Despliegue Final)

Descripción: Este bloque representa la fase operativa de explotación y despliegue práctico del modelo entrenado. Implementa una interfaz interactiva que permite conectar el

cuaderno de desarrollo con el almacenamiento local de la computadora del usuario. El motor de Inteligencia Artificial de VisioNet analiza la captura meteorológica en tiempo real mediante un pipeline de renderizado simétrico avanzado, diseñado especialmente para recalcular la escala de los elementos gráficos según la resolución nativa del archivo subido, consolidando la entrega del módulo de visión de Eco-Intelligence.

Acciones Principales del Script:

- **Carga de Pesos Óptimos y Resguardo (Fallback):** El sistema audita la presencia de los coeficientes optimizados del proyecto (`best.pt`). Si se detecta una anomalía en las rutas, el script activa un bloque de control de excepciones (*try-except*) que levanta la arquitectura base de YOLOv8 de manera segura, impidiendo la interrupción del servicio informático.
- **Interfaz de Captura de Datos (gdown / colab files):** Levanta la pasarela nativa `files.upload()` para transferir los archivos binarios de la imagen desde la PC del operador. El software se encarga de reestructurar el archivo bajo un estándar interno (`nueva_nube.jpg`) y purgar ejecuciones anteriores para evitar solapamientos en el disco virtual.
- **Inferencia y Parametrización Adaptativa de Pantalla:** Se ejecuta la predicción sobre la red profunda con un umbral técnico del 5% (`conf=0.05`). Al procesar las respuestas del modelo, el script lee dinámicamente las dimensiones de la imagen y recalcula en píxeles absolutos el espesor de las líneas y la escala del texto, garantizando lecturas nítidas tanto en imágenes de baja calidad como en resoluciones Ultra HD.
- **Algoritmo Antisuperposición de Etiquetas:** Implementa una solución de ordenamiento espacial en el eje de las abscisas. Cuando el modelo localiza estructuras nubosas muy continuas, el sistema procesa los índices y aplica una condicional de alternancia geométrica (`i % 2 != 0`). Esto desplaza los banners de texto de forma intercalada entre el techo superior y el suelo inferior de las cajas delimitadoras, impidiendo que las etiquetas se pisen entre sí y optimizando la interfaz gráfica.



PASO 3.3: Persistencia y Respaldo de Coeficientes Óptimos (MLOps)

Descripción: Este bloque de código implementa un protocolo estratégico de persistencia y resguardo de datos (*Data Backup*) dentro del ciclo de vida del desarrollo del modelo. Debido a que las instancias de computación distribuidas de Google Colab son de carácter volátil (los datos locales se eliminan de forma permanente tras la desconexión del entorno), el script se encarga de migrar los mejores pesos matemáticos generados por la red neuronal hacia la estructura de almacenamiento persistente en la nube corporativa de Google Drive.

Acciones Principales del Script:

- **Verificación y Creación Estructural de Directorios:** Mediante el uso del módulo nativo `os`, el sistema audita el direccionamiento hacia la ruta institucional del proyecto integrador. Si la carpeta de destino no es indexada en la unidad, el software ejecuta de forma transparente la instrucción recursiva `os.makedirs(..., exist_ok=True)` para asegurar la creación del contenedor sin interrumpir el flujo.
- **Validación de Integridad de Activos Técnicos:** El script ejecuta una comprobación lógica con `os.path.exists` para certificar que el archivo binario de pesos óptimos (`best.pt`) fue generado correctamente en el búfer temporal local tras finalizar el entrenamiento.
- **Transferencia y Sincronización Segura de Archivos:** A través del método de manipulación de archivos `shutil.copy`, el sistema clona la matriz de coeficientes entrenados y la exporta al almacenamiento en la nube bajo un nombre estandarizado (`best_nubes_yolov8.pt`). Esta operación garantiza que el equipo **VisioNet** disponga del modelo listo para fases de auditoría meteorológica, inferencia en vivo o integración en la estación **Eco-Intelligence**, mitigando el riesgo de pérdida de horas de cómputo tecnológico.

CONCLUSIÓN: Implementación de YOLOv8 en el Pipeline Meteorológico de VisioNet

Síntesis del Proyecto Integrador

La realización de este proyecto consolida de manera exitosa la convergencia entre el **Procesamiento Digital de Imágenes (PDI)** clásico y el aprendizaje profundo (*Deep Learning*). A través del diseño de un pipeline de software automatizado y estructurado, el equipo **VisioNet** logró transformar datos crudos de sensores visuales en un sistema de auditoría meteorológica inteligente y autónomo, pieza fundamental para la infraestructura de la estación **Eco-Intelligence**.

Hitos Técnicos Alcanzados

- **Optimización de la Ingesta de Datos:** La implementación de algoritmos de descompresión y mapeo dinámico en caché local garantiza un flujo de datos de alta velocidad, mitigando los cuellos de botella habituales en el almacenamiento en la nube distribuida.
- **Ingeniería de Características Críticas:** La aplicación de técnicas de PDI mediante **OpenCV** (conversión cromática, ecualización de histogramas para resaltar texturas nubosas y segmentación estructural con filtros Gaussianos y Canny) demostró que el preprocesamiento matemático es clave para limpiar el ruido atmosférico antes de alimentar cualquier red neuronal.
- **Robustez y Adaptabilidad en Inferencia (YOLOv8):** El entrenamiento bajo la metodología de transferencia de aprendizaje con la arquitectura YOLOv8 Nano demostró una eficiencia óptima para el cómputo perimetral. Los algoritmos defensivos añadidos (control de desbordamiento de bordes, detección autónoma de imágenes de fondo para evitar falsos positivos y la alternancia intercalada de etiquetas para prevenir colisiones gráficas) dotaron al sistema de una robustez de nivel de producción.
- **Persistencia y Gobernanza del Modelo (MLOps):** La automatización del resguardo y exportación estandarizada de los mejores pesos sinápticos ([best_nubes_yolov8.pt](#)) hacia unidades perennes asegura la replicabilidad del experimento y la optimización de los costos de cómputo del equipo.